

# Namespace Organization and Boundaries

---

**Series:** K8S | **Notebook:** 6 of 13 | **Created:** January 2026 | **Last Updated:** 02/05/2026

## Organizing Kubernetes Monitoring with Namespaces

---

Namespaces provide logical boundaries in Kubernetes for resource isolation, access control, and organizational structure. This notebook covers namespace strategies and how to leverage them in Dynatrace for filtered views, access control, and cost allocation.

---

## Table of Contents

---

1. [Namespace Strategies](#)
  2. [Namespace Monitoring in Dynatrace](#)
  3. [Resource Quotas and Limits](#)
  4. [Namespace-Based Access Control](#)
  5. [Multi-Tenant Clusters](#)
  6. [Cost Allocation by Namespace](#)
  7. [Namespace Best Practices](#)
-

# Prerequisites

| Requirement           | Details                                         |
|-----------------------|-------------------------------------------------|
| Dynatrace Environment | SaaS with Kubernetes monitoring                 |
| DynaKube              | Deployed with namespace selector configured     |
| Permissions           | entities.read , metrics.read                    |
| Knowledge             | K8S-01 Fundamentals, K8S-05 Workload Monitoring |

## 1. Namespace Strategies

### Common Namespace Patterns

| Strategy       | Structure                      | Use Case                      |
|----------------|--------------------------------|-------------------------------|
| By Environment | dev , staging , prod           | Single cluster, multiple envs |
| By Team        | team-checkout , team-catalog   | Team ownership                |
| By Application | app-frontend , app-backend     | Application isolation         |
| By Function    | monitoring , logging , ingress | Infrastructure services       |
| Hybrid         | team-checkout-prod             | Combined approach             |

### Namespace Naming Conventions

<prefix>-<owner>-<env>

Examples:

app-checkout-prod

team-platform-dev

infra-monitoring

## System Namespaces

| Namespace       | Purpose             | Monitor?       |
|-----------------|---------------------|----------------|
| kube-system     | Core K8s components | Yes (critical) |
| kube-public     | Public resources    | Minimal        |
| kube-node-lease | Node heartbeats     | No             |
| default         | Catch-all           | Discourage use |

```
// List all namespaces
fetch dt.entity.cloud_application_namespace
| fields entity.name, tags
| sort entity.name asc
| limit 50
```

## 2. Namespace Monitoring in Dynatrace

### DynaKube Namespace Selector

Control which namespaces receive OneAgent injection:

```
spec:
  cloudNativeFullStack: # namespaceSelector goes inside the deployment mode
    section
      namespaceSelector:
        matchLabels:
          monitoring: dynatrace
```

Or exclude specific namespaces:

```
spec:
  cloudNativeFullStack: # namespaceSelector goes inside the deployment mode
    section
      namespaceSelector:
        matchExpressions:
          - key: monitoring
            operator: NotIn
            values:
              - disabled
```

## Namespace Labels for Filtering

| Label       | Purpose            | Example              |
|-------------|--------------------|----------------------|
| team        | Team ownership     | team: checkout       |
| env         | Environment        | env: production      |
| cost-center | Billing allocation | cost-center: eng-123 |
| monitoring  | Injection control  | monitoring: enabled  |

```
// Namespace resource usage summary (CPU)
timeseries avgCpuUsageMillicores = avg(dt.kubernetes.container.cpu_usage),
from:-1h, by:{k8s.namespace.name}
| sort avgCpuUsageMillicores desc
| limit 15
```

```
// Memory usage by namespace
timeseries avgMemUsageBytes =
avg(dt.kubernetes.container.memory_working_set), from:-1h, by:
{k8s.namespace.name}
| sort avgMemUsageBytes desc
| limit 15
```

## 3. Resource Quotas and Limits

### ResourceQuota Monitoring

ResourceQuotas limit total resource consumption per namespace:

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: team-checkout
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
    limits.cpu: "20"
    limits.memory: 40Gi
    pods: "50"
```

## LimitRange for Defaults

```
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: team-checkout
spec:
  limits:
    - default:
        cpu: "500m"
        memory: "512Mi"
      defaultRequest:
        cpu: "100m"
        memory: "128Mi"
      type: Container
```

## Quota Utilization Metrics

| Metric             | Alert When        |
|--------------------|-------------------|
| CPU quota usage    | >80% of quota     |
| Memory quota usage | >80% of quota     |
| Pod count          | Approaching limit |

```
// CPU requests by namespace (quota tracking)
timeseries avgCpuRequests = avg(dt.kubernetes.workload.requests_cpu),
from:-1h, by:{k8s.namespace.name}
| sort avgCpuRequests desc
| limit 15
```

```
// Memory requests by namespace
timeseries avgMemRequests = avg(dt.kubernetes.workload.requests_memory),
from:-1h, by:{k8s.namespace.name}
| sort avgMemRequests desc
| limit 15
```

## 4. Namespace-Based Access Control

---

### Dynatrace IAM with Boundaries

Use Dynatrace boundaries to restrict visibility by namespace:

**Policy Example (Grant access to checkout namespace):**

```
{
  "name": "pol-namespace-checkout-viewer",
  "statement": {
    "effect": "ALLOW",
    "permissions": ["environment:roles:viewer"],
    "conditions": [
      {
        "kubernetes.namespace": "checkout"
      }
    ]
  }
}
```

### Segment by Namespace

Create segments for namespace-based filtering:

| Segment        | Filter                                         | Use Case      |
|----------------|------------------------------------------------|---------------|
| seg-checkout   | k8s.namespace.name == "checkout"               | Team view     |
| seg-production | k8s.namespace.name matches "*-prod"            | Prod only     |
| seg-platform   | k8s.namespace.name in ("monitoring","logging") | Platform team |

## Kubernetes RBAC Alignment

Align Dynatrace access with Kubernetes RBAC:

| K8s Role         | Dynatrace Access          |
|------------------|---------------------------|
| namespace-admin  | Full namespace visibility |
| namespace-viewer | Read-only namespace data  |
| cluster-admin    | All namespaces            |

## 5. Multi-Tenant Clusters

### Tenant Isolation Strategies

| Level   | Mechanism                 | Dynatrace Support    |
|---------|---------------------------|----------------------|
| Soft    | Namespace + NetworkPolicy | Boundaries, segments |
| Hard    | Separate clusters         | Separate DynaKubes   |
| Virtual | vCluster                  | Namespace labels     |

### DynaKube per Tenant (Hard Isolation)

For strong isolation, use separate DynaKube instances:

```
# Tenant A
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube-tenant-a
  namespace: dynatrace
spec:
  apiUrl: https://tenant-a.live.dynatrace.com/api
  namespaceSelector:
    matchLabels:
      tenant: a
```

## Shared DynaKube (Soft Isolation)

Single DynaKube with boundary-based access control:

```
spec:
  oneAgent:
    cloudNativeFullStack: {}
  # All namespaces monitored, access controlled via IAM
```

```
// Workload count by namespace
fetch dt.entity.cloud_application
| summarize workloadCount = count()
| limit 20
```

## 6. Cost Allocation by Namespace

### Resource-Based Cost Allocation

Calculate costs based on resource consumption:

| Resource       | Cost Metric | Calculation                 |
|----------------|-------------|-----------------------------|
| <b>CPU</b>     | Core-hours  | CPU usage × hours × rate    |
| <b>Memory</b>  | GB-hours    | Memory usage × hours × rate |
| <b>Storage</b> | GB-hours    | PVC size × hours × rate     |



# Labels for Cost Centers

```
apiVersion: v1
kind: Namespace
metadata:
  name: team-checkout
  labels:
    cost-center: "eng-checkout-123"
    team: "checkout"
    budget-owner: "alice@example.com"
```

```
// Resource consumption by namespace (for cost allocation)
timeseries avgCpuMillicores = avg(dt.kubernetes.container.cpu_usage),
from:-1h, by:{k8s.namespace.name}
| sort avgCpuMillicores desc
| limit 15
```

## 7. Namespace Best Practices

### Naming and Organization

| Practice                             | Reason                           |
|--------------------------------------|----------------------------------|
| Use consistent naming                | Easier filtering and automation  |
| Apply standard labels                | Cost allocation, access control  |
| Avoid <code>default</code> namespace | Encourages explicit organization |
| Document ownership                   | Clear responsibility             |

### Resource Management

| Practice              | Reason                      |
|-----------------------|-----------------------------|
| Set ResourceQuotas    | Prevent resource hogging    |
| Configure LimitRanges | Ensure defaults             |
| Monitor quota usage   | Proactive capacity planning |

## Security

| Practice                     | Reason                  |
|------------------------------|-------------------------|
| Apply NetworkPolicies        | Namespace isolation     |
| Use RBAC per namespace       | Least privilege         |
| Separate sensitive workloads | Compliance requirements |

## Monitoring

| Practice                          | Reason                 |
|-----------------------------------|------------------------|
| Label namespaces for filtering    | Easy DQL queries       |
| Create namespace-based dashboards | Team-specific views    |
| Set up namespace alerts           | Targeted notifications |

## Next Steps

With namespace organization in place, proceed to:

| Next Notebook                     | Topic                     |
|-----------------------------------|---------------------------|
| <b>K8S-07: Events and Logs</b>    | Kubernetes event analysis |
| <b>K8S-08: DQL for Kubernetes</b> | Advanced query patterns   |
| <b>K8S-09: Troubleshooting</b>    | Debugging K8s monitoring  |

## Summary

In this notebook, you learned:

- Namespace strategies and naming conventions
- DynaKube namespace selector configuration

- Resource quota and limit monitoring
  - Namespace-based access control with boundaries
  - Multi-tenant cluster patterns
  - Cost allocation by namespace
  - Namespace best practices
- 

## References

---

- [Kubernetes Namespaces](#)
  - [Resource Quotas](#)
  - [Dynatrace Boundaries](#)
- 

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*